

Summary

Recursion often confuses new programmers, but it is built on the simple idea that a function can call itself. Every time a function call is made, a new frame object with information related to the call (such as local variables and a return address for the execution to move to when the function returns) is added to the call stack. The call stack, being a stack data structure, can be altered only by having data added to or removed from its “top.” This is called *pushing to* and *popping from* the stack, respectively.

The call stack is handled by the program implicitly, so there is no call stack variable. Calling a function pushes a frame object to the call stack, and returning from a function pops a frame object from the call stack.

Recursive functions have recursive cases, those in which a recursive call is made, and base cases, those where the function simply returns. If there is no base case or a bug prevents a base case from being run, the execution causes a stack overflow that crashes the program.

Recursion is a useful technique, but recursion doesn’t automatically make code “better” or more “elegant.” This idea is explored more in the next chapter.